

Motion Planning in Static and Dynamic Environments

1st Armon Barakchi
 abarakchi@ucsd.edu
 A16950466

I. INTRODUCTION

In robotics, motional planning is a fundamental problem that comes after perception. The basic setup is as follows: given a start configuration, a goal configuration, and a set of obstacles, we wish to compute a collision-free path through the environment while optimizing some notion of cost.

The algorithms that perform this crucial task fall into two main categories: search-based methods and sampling-based methods. Search-based methods discretize the configuration space to construct a graph and solve a *deterministic shortest path (DSP)* problem over that graph. In contrast, sampling-based planners incrementally construct a collision-free tree through random sampling of the free space. The most crucial contrast is that sampling-based algorithms forego graph construction, leading to potentially faster motion planning.

This work studies the performance of both search-based and sampling-based planning algorithms in static and dynamic three-dimensional environments. Namely, Weighted A*, Any-time Repairing A* (ARA*), RRT*, and RRT-Connect are compared across the environments. The environments vary in terms of geometry and complexity. Additionally, the dynamic environments rotate the goal position over time under strict planning deadlines, which introduces additional emphasis on replanning speed.

The results of this work demonstrate a clear trade-off between optimality and speed. Search-based methods perform consistently well in complex environments where optimality is necessary, while sampling-based methods perform well in open spaces where speed is necessary.

II. PROBLEM FORMULATION

The general setting for the environments is as follows: we consider the motion planning problem for a point robot in a bounded 3-D Euclidean workspace $\mathcal{W} \subset \mathbb{R}^3$. The workspace has a rectangular boundary defined as: $\mathcal{B} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}]$ and a set of M rectangular obstacle blocks $\{\mathcal{O}_k\}_{k=1}^M$. Each obstacle is an axis-aligned bounding box (AABB) specified by its lower-left corner $(x_{\min}^k, y_{\min}^k, z_{\min}^k)$ and upper-right corner $(x_{\max}^k, y_{\max}^k, z_{\max}^k)$.

A. Configuration Space

Since the robot is modeled as a point mass with 3 translational degrees of freedom and no rotational constraints the

configuration space is just the workspace: $\mathcal{C} = \mathcal{W} \subset \mathbb{R}^3$ [1]. The **free space** and **obstacle space** are defined as:

$$\mathcal{C}_{\text{free}} = \mathcal{B} \setminus \bigcup_{k=1}^M \mathcal{O}_k, \quad (1)$$

$$\mathcal{C}_{\text{obs}} = \mathcal{C} \setminus \mathcal{C}_{\text{free}}. \quad (2)$$

A configuration $\mathbf{x} \in \mathcal{C}$ is *collision-free* if and only if $\mathbf{x} \in \mathcal{C}_{\text{free}}$.

B. Deterministic Shortest Path Formulation

Formally, motion planning is formulated as a **Deterministic Shortest Path (DSP)** problem on a graph [2]. The free space $\mathcal{C}_{\text{free}}$ is discretized by overlaying a uniform 3-D grid of resolution δ to construct a graph $G = (\mathcal{V}, \mathcal{E}, \mathcal{C})$, where each node $i \in \mathcal{V}$ corresponds to a grid point $\mathbf{x}_i \in \mathcal{C}_{\text{free}}$ and graph edges connect each node to its 26-connected neighbors.

The edge weight, or cost-to-go between adjacent nodes i and j is the Euclidean distance between their configurations:

$$c_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|_2, \quad \forall (i, j) \in \mathcal{E}. \quad (3)$$

An edge (i, j) is included in $\mathcal{E}$ only if the line segment $\overline{\mathbf{x}_i \mathbf{x}_j}$ lies entirely within $\mathcal{C}_{\text{free}}$. This is verified in continuous space using segment-AABB intersection testing via the slab method [3], ensuring collision checking is exact. Since all edge weights satisfy $c_{ij} \geq 0$, the graph contains no negative cycles, which is a necessary condition for this problem to be well-posed.

A path is defined as a sequence of nodes $i_{1:q} := (i_1, i_2, \dots, i_q)$ with $i_k \in \mathcal{V}$, $i_1 = s$, and $i_q = \tau$. The path length is the sum of edge weights along the path [2]:

$$J^{i_{1:q}} = \sum_{k=1}^{q-1} c_{i_k, i_{k+1}}. \quad (4)$$

The set of all paths from start to goal is denoted $\mathcal{P}_{s, \tau} = \{i_{1:q} \mid i_k \in \mathcal{V}, i_1 = s, i_q = \tau\}$. The objective of motion planning is to find a path of minimum length:

$$\text{dist}(s, \tau) = \min_{i_{1:q} \in \mathcal{P}_{s, \tau}} J^{i_{1:q}}, \quad i_{1:q}^* \in \arg \min_{i_{1:q} \in \mathcal{P}_{s, \tau}} J^{i_{1:q}}. \quad (5)$$

C. Static Goal Setting

There are 5 static environments (E1–E5), where the goal and obstacles are stationary. In these maps, the formulation is exactly as presented in Section II-B and we seek to find a minimum cost path from start to goal. A successful path is one such that the goal is reached and there are no collisions.

D. Dynamic Goal Setting

For Environments E6 and E7, the goal moves through a sequence of 8 positions, remaining at each position for a time interval t_n before transitioning. The planner must compute a collision-free path from the fixed start s to the current goal $\tau^{(n)}$ within t_n , without foreknowledge of future goal positions. At each step n the same DSP objective applies:

$$i_{1:q}^{*(n)} \in \arg \min_{i_{1:q} \in \mathcal{P}_{s, \tau^{(n)}}} J^{i_{1:q}}, \quad T_{\text{plan}}^{(n)} \leq t_n, \quad (6)$$

where $T_{\text{plan}}^{(n)}$ is the time to compute the n -th path. Since the graph G does not change between goals, the cost-to-arrive estimates $\{g_i\}$ computed for goal $\tau^{(n)}$ remain partially valid when the goal changes to $\tau^{(n+1)}$.

A successful run on the dynamic maps is defined as having reached all the goals without collisions in the prescribed amount of time.

III. TECHNICAL APPROACH

A. Collision Checking

Collision checking is performed in continuous space using the slab method [3]. Each edge of the planned path is treated as a line segment and tested against every obstacle.

1) *Slab Method*: Each obstacle $\mathcal{O}_k$ is represented as the intersection of three pairs of parallel planes, one pair per coordinate axis. These are the "slabs."

$$\begin{aligned} \mathcal{O}_k = & \{x_{\min}^k \leq x \leq x_{\max}^k\} \\ & \cap \{y_{\min}^k \leq y \leq y_{\max}^k\} \\ & \cap \{z_{\min}^k \leq z \leq z_{\max}^k\}. \end{aligned} \quad (7)$$

A segment between endpoints $\mathbf{A}, \mathbf{B} \in \mathbb{R}^3$ is parameterized as:

$$\mathbf{P}(t) = \mathbf{A} + t(\mathbf{B} - \mathbf{A}), \quad t \in [0, 1]. \quad (8)$$

For each axis $i \in \{x, y, z\}$, the values at which $\mathbf{P}(t)$ enters and exits slab i are:

$$t_1^i = \frac{\mathcal{O}_{\min}^{k,i} - A_i}{B_i - A_i}, \quad t_2^i = \frac{\mathcal{O}_{\max}^{k,i} - A_i}{B_i - A_i}, \quad (9)$$

with $t_{\text{enter}}^i = \min(t_1^i, t_2^i)$ and $t_{\text{exit}}^i = \max(t_1^i, t_2^i)$. The global entry and exit parameters across all three slabs are:

$$t_{\text{enter}} = \max_i t_{\text{enter}}^i, \quad t_{\text{exit}} = \min_i t_{\text{exit}}^i. \quad (10)$$

Finally, the segment intersects the obstacle if and only if:

$$t_{\text{enter}} \leq t_{\text{exit}} \quad \text{and} \quad t_{\text{enter}} \leq 1 \quad \text{and} \quad t_{\text{exit}} \geq 0. \quad (11)$$

If the segment being tested is parallel to slab i (i.e. $|B_i - A_i| < \epsilon$ for a small tolerance $\epsilon = 10^{-12}$), the slab method reduces seeing whether $A_i \in [\mathcal{O}_{\min}^{k,i}, \mathcal{O}_{\max}^{k,i}]$. If it is not, then intersection is rejected for that obstacle.

2) *Vectorised Implementation*: Since, doing the above operation for each slab individually is too expensive, we check each segment against all M obstacles simultaneously. To do this, the computation is vectorized across the full block array. The entry and exit parameters are represented as vectors $\mathbf{t}_{\text{enter}}, \mathbf{t}_{\text{exit}} \in \mathbb{R}^M$, and updated per axis. A boolean mask tracks which boxes remain in contention, and any box where $t_{\text{enter}} > t_{\text{exit}}$ is eliminated.

3) *Full Path Checking*: The full path collision checking is performed after a path has been calculated. A complete path is validated by iterating over each consecutive segment (i_k, i_{k+1}) and using the method described in Section III-A2. Additionally, all waypoints are checked against the workspace boundary $\mathcal{B}$. The collision checker returns whether the path is safe or not.

B. Search-Based Planning

Since search-based planning algorithms traverse a graph, we must discretize $\mathcal{C}_{\text{free}}$ into a finite graph $G = (\mathcal{V}, \mathcal{E}, \mathcal{C})$. This will give us a DSP problem, which is a well-studied problem with many well-studied algorithms to find solutions. These algorithms are resolution complete, meaning that if a feasible path exists at the given grid resolution δ , the algorithm is guaranteed to find it in finite time [4].

1) *Graph Construction*: The free space $\mathcal{C}_{\text{free}}$ is discretized by superimposing a uniform 3-D grid of resolution δ over the workspace boundary $\mathcal{B}$. Each grid node $i \in \mathcal{V}$ is mapped to and from world coordinates via:

$$\mathbf{g}_i = \left\lfloor \frac{\mathbf{x}_i}{\delta} \right\rfloor \in \mathbb{Z}^3, \quad \mathbf{x}_i = \delta \cdot \mathbf{g}_i. \quad (12)$$

Each node is connected to its **26-connected neighbours**, corresponding to all non-zero translations: $\Delta \mathbf{g} \in \{-1, 0, 1\}^3 \setminus \{0\}$. This gives three types of neighbors for any given node:

- **Face neighbours** (6): one non-zero component, $c_{ij} = \delta$.
- **Edge neighbours** (12): two non-zero components, $c_{ij} = \sqrt{2}\delta$.
- **Corner neighbours** (8): three non-zero components, $c_{ij} = \sqrt{3}\delta$.

All 26 offsets and their costs are pre-computed once as the `_OFFSETS` list and `_COSTS` dictionary. An edge (i, j) is included in the graph if and only if the segment $\overline{\mathbf{x}_i \mathbf{x}_j}$ passes the collision check.

a) *Scaling to large maps.*: Search-based planning algorithms can be memory intensive due to graph construction, so we have implemented two mechanisms to make this computationally feasible. Firstly, the graph is constructed lazily during search, i.e. no nodes or edges are pre-allocated. A node only enters the cost table when it is first reached by an edge expansion. Because of this, memory is proportional to the number of nodes actually reached, not the total grid size $O((L/\delta)^3)$. To mitigate the potential issue of expanding too many nodes, which would still create an extremely large graph, we have used heuristic inflation by making $\varepsilon > 1$ which drastically reduces the number of node expansions. Together, these methods prevent the graph from becoming millions of

nodes large in complicated environments, while still being able to maintain a fine resolution relative to the map size.

b) Memory complexity.: The search maintains three data structures: a cost table `g_score`, a parent pointer table `parent`, and a closed set `closed`, each storing at most $|\mathcal{V}_{\text{exp}}|$ (the number of nodes expanded). The min-heap OPEN list holds at most $O(|\mathcal{V}_{\text{exp}}|)$ entries.

2) *Weighted A** [4]: One approach to the DSP problem on a graph is *Weighted A**, which is a label-correcting algorithm. Each visited node i has a cost-to-arrive estimate $g(i)$, which is initialized as $g(s) = 0$ and $g(i) = \infty$ for all other nodes.

*Weighted A** solves the DSP problem on the graph as a label correcting algorithm. Each visited node i maintains a cost-to-arrive estimate $g(i)$, initialised as $g(s) = 0$ and $g(i) = \infty$ for all other nodes. At each iteration, the node with the lowest evaluation function is popped from OPEN. The 26 neighbors of the popped node are relaxed: if $g(i) + c_{ij} < g(j)$, then $g(j)$ is corrected to become $g(j) = g(i) + c_{ij}$ and j is pushed onto OPEN. This step is known as "label correction" prioritized by the evaluation function. More formally:

$$f(i) = g(i) + \varepsilon \cdot h(i), \quad (13)$$

where $h(i) = \|\mathbf{x}_i - \mathbf{x}_\tau\|_2$ is the Euclidean distance heuristic and $\varepsilon \geq 1$ is the suboptimality weight which dictates the tradeoff between speed and optimality.

*Weighted A** is different from standard *A** because it inflates the heuristic function by a factor ε , which biases node selection more strongly towards the goal. This reduces the number of expansions, but crucially, it can result in path-length suboptimality. Standard *A** expands nodes in order of the true f -value and will return the shortest path every time.

In my implementation, the OPEN list is a Python `heapq` min-heap ordered by $f(i)$. Since `heapq` doesn't have priority updates, duplicate entries are inserted into the OPEN list. Stale entries are discarded lazily by checking if the node is in the CLOSED list upon popping. Goal termination is checked in world space: the search terminates when the popped node satisfies $\|\mathbf{x}_i - \mathbf{x}_\tau\|_2 \leq 1.5\delta$. The path is made by following parent pointers back to the start node and adding the exact goal coordinate.

Since all edge costs satisfy $c_{ij} \geq 0$ and $h(i)$ is admissible, the algorithm produces a **deterministic, finite-time** solution with a hard suboptimality bound:

$$J^* \leq J_\varepsilon \leq \varepsilon \cdot J^*, \quad (14)$$

where J^* is the shortest path length on the graph and J_ε is the returned length. Setting $\varepsilon = 1$ recovers standard *A** with $J_\varepsilon = J^*$. Note that J^* is optimal on the discretized graph; the true continuous optimum may be shorter depending on δ . With the fine resolution capable by the lazy graph construction, J^* is likely a good approximation of the true shortest path. Additionally, *Weighted A** is **resolution complete**: if a path exists on the grid, it is guaranteed to be found.

3) *Anytime Repairing A* (ARA*)* [5]: *ARA** is an extension of *Weighted A** that also utilizes label correcting, but produces multiple trajectories within a given time budget. Rather than using a fixed ε like *Weighted A**, *ARA** begins with a large $\varepsilon_{\text{init}}$. This enforces a quick initial feasible solution that may be suboptimal. The algorithm then iteratively decreases ε by a fixed step $\Delta\varepsilon$ and repairs the search until either $\varepsilon = 1$ (standard *A**) or the time limit T_{limit} is reached.

*ARA** differs from standard *A** in two ways. Firstly, and most importantly, ε is not fixed, which enables an initial coarse search that becomes more fine as time progresses. Secondly, the `g_score` and `parent` dictionaries persist across iterations, meaning that only the OPEN list and `closed` set are reinitialized at each iteration. Nodes whose g -values are already consistent under the tighter ε are settled immediately upon re-expansion without traversing the graph again. This means each repair pass re-expands only nodes that benefit from the tighter bound. This is analagous to an incremental repair.

In my implementation, the OPEN list for iteration $k = 1$ is seeded with the start node using its g -value from the previous iteration:

$$\text{OPEN}_{k+1} = \{(g(s) + \varepsilon_{k+1} \cdot h(s), g(s), s)\}, \quad (15)$$

where $g(s) = 0$ always. The time limit T_{limit} is enforced by checking `time.time()` at every node expansion, returning the best solution found so far upon timeout.

*ARA** produces a **deterministic** solution at every iteration k with a monotonically tightening suboptimality bound:

$$J^* \leq J_{\varepsilon_k} \leq \varepsilon_k \cdot J^*, \quad \varepsilon_k = \varepsilon_{\text{init}} - k \cdot \Delta\varepsilon. \quad (16)$$

The sequence $J_{\varepsilon_0} \geq J_{\varepsilon_1} \geq \dots \geq J_{\varepsilon_K}$ is monotonically non-increasing. If $\varepsilon = 1$, the solution satisfies $J_{\varepsilon_K} = J^*$. These guarantees are deterministic and hold at every iteration. Additionally, similar to *Weighted A**, *ARA** is resolution complete at every $\varepsilon_k \geq 1$: if a path exists on the grid, it is found in finite time at the first iteration.

C. Sampling-Based Planning

Search-based planning is only one type of motion planning in a continuous environment. Another approach is using *Sampling-based planning*, which solves the motion planning problem by incrementally building a collision-free tree by randomly sampling $\mathcal{C}_{\text{free}}$. The graph $G = (\mathcal{V}, \mathcal{E})$ is a tree rooted at the start with sampled configurations as nodes and collision-free segments as edges. The DSP problem is solved by growing this tree until a node within reach of the goal is found. Compared to search-based methods, sampling-based algorithms are able to scale more favorably in memory, but suffer from suboptimality.

1) *RRT** [6]: *RRT** extends basic RRT by incorporating a two-step rewiring procedure that introduces cost minimization into the tree. This addition gives it asymptotic optimality guarantees. Basic Rapidly-Exploring Random Tree (RRT) connects each new node to its nearest existing node and never fixes

these connections, even if a more favorable one appears. RRT* replaces with the following two step process:

- 1) **Best-Parent Selection**, which connects the new node to the lowest-cost reachable neighbour within radius r .
- 2) **Rewiring**, which updates the parents of existing neighbors if routing through the new node reduces their cost.

These steps are analogous to the label correction step in the search-based planning algorithms.

The algorithm is as follows: at each iteration a sample $\mathbf{x}_{\text{rand}}$ is drawn uniformly from $\mathcal{C}_{\text{free}}$, with probability p_{goal} of sampling the goal directly. The nearest tree node is found by:

$$\mathbf{x}_{\text{nearest}} = \arg \min_{\mathbf{x} \in \mathcal{V}} \|\mathbf{x} - \mathbf{x}_{\text{rand}}\|_2, \quad (17)$$

and a new node is obtained by steering at most η toward the sample. If the connecting segment is collision-free, the best parent within radius r is selected:

$$\mathbf{x}_{\text{parent}} = \arg \min_{\mathbf{x} \in \mathcal{X}_{\text{near}}} [g(\mathbf{x}) + \|\mathbf{x} - \mathbf{x}_{\text{new}}\|_2], \quad (18)$$

where $\mathcal{X}_{\text{near}} = \{\mathbf{x} \in \mathcal{V} \mid \|\mathbf{x} - \mathbf{x}_{\text{new}}\|_2 \leq r\}$. The rewiring step then checks every $\mathbf{x} \in \mathcal{X}_{\text{near}}$:

$$\begin{aligned} &\text{if } g(\mathbf{x}_{\text{new}}) + \|\mathbf{x}_{\text{new}} - \mathbf{x}\|_2 < g(\mathbf{x}) \\ &\text{and the segment is collision-free, then} \\ &\text{parent}(\mathbf{x}) \leftarrow \mathbf{x}_{\text{new}}. \end{aligned} \quad (19)$$

The nearest node query (Equation 17) is a linear scan over all nodes in the tree, meaning the time complexity is $O(|\mathcal{V}|)$ per iteration. Free-space samples are drawn by uniform sampling within the boundaries and we reject configurations that collide with any obstacle. This process is retried 1000 times. The rewiring radius is a hyperparameter that is tuned beforehand.

RRT* is probabilistically complete and asymptotically optimal, meaning that as $n \rightarrow \infty$, the cost of the best path converges to the continuous space optimum J^* almost surely. However, this algorithm's drawback is that for any finite n , there is no hard suboptimality guarantee like A*-family algorithms.

2) *RRT-Connect* [7]: RRT-connect is a bidirectional variation of standard RRT. This algorithm grows two trees simultaneously — $\mathcal{T}_a$ from $\mathbf{x}_s$ and $\mathcal{T}_b$ from $\mathbf{x}_t$ and tries to connect them after each extension.

While basic RRT grows a single tree from the start and checks proximity to the goal at each step, RRT-Connect instead grows two trees and uses a greedy connect procedure that repeatedly extends $\mathcal{T}_b$ toward the latest node in $\mathcal{T}_a$ until it either reaches it or is blocked. This strategy reduces the number of samples needed to bridge the start and goal, proving particularly effective in open environments. Unlike RRT*, there is no rewiring and the tree structure is determined purely by the extension order.

The algorithm proceeds as follows: each EXTEND call attempts to make a single step toward a target and returns one of three status codes: TRAPPED (blocked by collision), ADVANCED (step taken, target not reached), or REACHED (target reached within tolerance 10^{-6} m). (Connect) and (Extend) are

looped until the status code is no longer (Advanced). After each extension of $\mathcal{T}_a$, the trees are swapped so that both can grow at almost equal rates. When (Connect) returns (Reached), the path is extracted by tracing parent points from the meeting node to the root of each tree, reversing $\mathcal{T}_b$'s segment, and combining.

RRT-Connect is probabilistically complete: as $n \rightarrow \infty$, it finds a feasible path with probability 1 if one exists. However, unlike RRT*, it is not asymptotically optimal because there is no rewiring. The returned path is the first feasible path found by the bidirectional search, whose quality depends entirely on the random samples drawn.

D. Final Technical Approach

In this work, all four algorithms are compared across all the static maps, and the A*-family algorithms are compared in the dynamic maps. These algorithms are evaluated based on their performance.

a) *Static environments (E1–E5)*.: For each static map, all four algorithms are run from the same start and goal configuration.

b) *Dynamic environments (E6–E7)*.: For each dynamic map, ARA* and Weighted A* are evaluated to find the minimum time budget (t_1, t_2) for success. For each goal $n \in \{1, \dots, 8\}$, we record whether the planner successfully found a collision-free path within the time budget, the path length $J^{(n)}$, and the planning time $T_{\text{plan}}^{(n)}$. A goal is marked as failed if the planner does not return a feasible path within t_n seconds. Overall success requires all 8 goals to be reached within their respective time budgets with no collisions.

Overall, this work aims to compare these different algorithms in each environment.

IV. RESULTS

The results for each environment are shared below. See Appendix A for visualizations of each environment.

A. Static Environments

For the static environments, the algorithms are evaluated for performance based on:

- 1) Quality of computed paths: this is measured as the relative suboptimality J/J_δ^* with respect to the grid-optimal path length at any given value of δ .
- 2) Computational efficiency: for search-based algorithms, this is quantified by the number of nodes expanded from the OPEN list; for sampling-based algorithms, this is quantified by the number of tree nodes added before a solution is found (i.e. the number of paths sampled). This is also evaluated based on runtime for both types of algorithms.
- 3) The effect of algorithm-specific parameters: the choice of heuristic function for Weighted A* and ARA*, and the choice of η , r , p_{goal} and iteration budget n_{max} for RRT*.

The quality of the computed paths is quantified through the ratio of the returned path length of any given motion

planning method to the optimal path length J_δ^* obtained from a Weighted A* search with $\varepsilon = 1$ on the discretized graph. Although sampling-based methods do not operate on this grid, the comparison is still meaningful because J_δ^* provides a valid upper-bound on the true continuous space optimum since $J_{\text{cont}}^* \leq J_\delta^*$. Additionally, at $\delta = 0.5$ m, the discretization gap is small relative to the total path lengths, further enforcing that this metric is meaningful.

Below, Table I shares the parameters used for all static environments in the base cases, leading to the results in Table II. Note, that for this table, the heuristic for the A* family algorithms is the Euclidean distance.

The paths planned in each environment by each algorithm can be found in Appendices B–F. Note that for sampling-based algorithms, these are not the only possible paths that could be planned, just a representative example of what happens most of the time. By that, we mean that when a certain algorithm produces a failure most of the time in a certain environment, that is what will be presented in the appendix.

TABLE I: Parameters used for all static Environments.

Algorithm	Parameter	Value	Description
Weighted A*	ε	1.5	Heuristic inflation weight
	δ	0.5	Grid resolution (m)
ARA*	$\varepsilon_{\text{init}}$	5.0	Initial heuristic weight
	$\varepsilon_{\text{final}}$	1.0	Final heuristic weight
	$\Delta\varepsilon$	0.5	Weight reduction step
	T_{limit}	5.0 s	Anytime time budget
	δ	0.5	Grid resolution (m)
RRT*	n_{max}	2000	Maximum iterations
	η	0.3	Step size (m)
	r	1.5	Rewiring radius (m)
	p_{goal}	0.1	Probability of sampling goal
RRT-Connect	n_{max}	3000	Maximum iterations
	η	0.5	Step size (m)

1) **Environment 1 – Single Cube:** With the parameters in table I, and the euclidean distance heuristic for A*-family algorithms, the results in Table II were accomplished. Note that $J_{\delta=0.5}^* = 7$.

For this simple environments, all four algorithms succeed. Weighted A* and ARA* achieve optimal paths very quickly because of the relatively short path. The sampling-based algorithms also succeed, but with varying speed. RRT* is much slower due to the rewiring step cross 2000 iterations. Additionally, this algorithm failed to converge to the optimal path in the iteration budget, so it kept going which took a while. In contrast, RRT-connect is by the far the fastest algorithm, requiring only one sample from each tree, which, again, reflects the open nature of this environment. Both sampling-based algorithms return a path of length 8, which is a suboptimality ratio of 1.143 relative to the grid-optimal solution.

2) **Environment 2 – Maze:** With the parameters in table I, and the euclidean distance heuristic for A*-family algorithms, the results in Table II were accomplished. Note that $J_{\delta=0.5}^* = 79$.

This is the most challenging static environment. Weighted A* and ARA* both succeed, returning a path length of

81 and taking approximately the same amount of time at three seconds each. The high node expansion counts for each algorithm reflect the complexity of this environment, which forces the search to explore many nodes before finding the goal. The similar expansion counts between the two algorithms suggest that ARA* did not benefit from g -value reuse here significantly.

Both sampling-based algorithms fail entirely. Neither were able to return a feasible path within the iteration budget prescribed here. This reflects the complexity of the map and points to a bigger issue with sampling-based algorithms: The narrow passage problem. Since the maze corridors occupy a small fraction of the total environment volume, it is extremely unlikely for random samples to thread through the correct sequence of corridors.

3) **Environment 3 – Flappy Bird:** With the parameters in table I, and the euclidean distance heuristic for A*-family algorithms, the results in Table II were accomplished. Note that $J_{\delta=0.5}^* = 25$.

In this environment, all four algorithms succeed. Weighted A* returns the optimal path, but in a slightly longer time than the suboptimal path that is returned by ARA*. Despite its severe suboptimality, RRT-connect succeeds in the shortest about of time. Finally, RRT*, while still returning a more optimal path than RRT-connect, takes significantly more time to run than any of the other algorithms, again revealing its weaknesses in geometrically-constrained environments. This environment illustrates the speed-quality trade-off: RRT-Connect is fastest but least optimal; Weighted A* is the third slowest but optimal.

4) **Environment 4 – Monza:** With the parameters in table I, and the euclidean distance heuristic for A*-family algorithms, the results in Table II were accomplished. Note that $J_{\delta=0.5}^* = 78$.

Weighted A* and ARA* both succeed, returning the optimal path length of 78 with similar runtimes and node expansions. Unsurprisingly, both sampling-based algorithms fail here due to the tight corridors around each obstacle – again revealing the shortcomings of sampling-based algorithms in environments with narrow passages.

5) **Environment 5 – Tower:** With the parameters in table I, and the euclidean distance heuristic for A*-family algorithms, the results in Table II were accomplished. Note that $J_{\delta=0.5}^* = 32$.

Weighted A* succeeds in 0.75 s with 1626 nodes expanded, returning the optimal path length of 32. ARA* is much faster at 0.20 s with only 310 nodes expanded, returning a path of length 37. The significant reduction in expanded nodes for ARA* demonstrates the performance boost from starting with a higher ε and returning early. RRT* fails completely again. However, RRT-connect works fairly well here. Despite return the least optimal path out of all successful algorithms, it does so relatively fast. RRT-Connect succeeds here where RRT* does not, likely because its bidirectional strategy allows $\mathcal{T}_b$ rooted at the elevated goal to grow downward and meet $\mathcal{T}_a$ partway, reducing the effective search distance.

TABLE II: Performance comparison of planning algorithms across all environments.

Environment	Algorithm	Success	Runtime (s)	Path Dist.	Nodes Expanded	Samples	Optimality
Env. 1	Weighted A*	TRUE	5.62×10^{-3}	7	10	–	1
	ARA*	TRUE	7.28×10^{-3}	7	10	–	1
	RRT*	TRUE	4.81	8	–	2000	1.143
	RRT-Connect	TRUE	7.64×10^{-4}	8	–	1	1.143
Env. 2	Weighted A*	TRUE	3.04	81	6975	–	1.025
	ARA*	TRUE	3.03	81	6882	–	1.025
	RRT*	FALSE	0.364	N/A	–	2000	N/A
	RRT-Connect	FALSE	1.22	N/A	–	3000	N/A
Env. 3	Weighted A*	TRUE	0.360	25	1033	–	1
	ARA*	TRUE	0.231	27	634	–	1.08
	RRT*	TRUE	2.017	29	–	2000	1.16
	RRT-Connect	TRUE	0.166	37	–	925	1.48
Env. 4	Weighted A*	TRUE	0.859	78	2725	–	1
	ARA*	TRUE	0.869	78	2716	–	1
	RRT*	FALSE	1.001	N/A	–	2000	N/A
	RRT-Connect	FALSE	1.180	N/A	–	3000	N/A
Env. 5	Weighted A*	TRUE	0.754	32	1626	–	1
	ARA*	TRUE	0.195	37	310	–	1.156
	RRT*	FALSE	0.547	N/A	–	2000	N/A
	RRT-Connect	TRUE	0.608	43	–	2013	1.344

6) **Parameter Variation:** Due to the poor performance of RRT* in the narrow passage environments, we present some parameter variations for Environments 2, 4, and 5 to see if it is possible to successfully motion plan in these settings using an asymptotically optimal sampling-based algorithm. Below, Table III describes the new parameters used for each environment. The visualizations of these paths are presented in Appendix I

TABLE III: Augmented RRT* Parameters for attempts at successful narrow passage navigation.

Algorithm	Parameter	Value	Environment
RRT*	$n_{\max}$	500000	Environment 2
	η	0.1	
	r	0.5	
	p_{goal}	0.3	
RRT*	$n_{\max}$	200000	Environment 4
	η	0.25	
	r	0.5	
	p_{goal}	0.3	
RRT*	$n_{\max}$	10000	Environment 5
	η	0.25	
	r	0.1	
	p_{goal}	0.2	

Below, Table IV describes the performance of RRT* across these narrow passage environments.

For environments 2 and 4, despite extremely high iteration counts and small step sizes, the RRT* algorithm still could not find a successful path from start to goal. For environment 5, moderate max iteration count increase led to a successful path. However, the quality of this successful path was quite poor. The planning time was significantly longer than all other successful algorithms from Table II, and in terms of optimality, it was also the worst. This result displays the shortcoming of unidirectional sampling-based algorithms in narrow passages,

TABLE IV: RRT* Performance Across Selected Environments with augmented parameters.

Env.	Success	Runtime (s)	Path Dist.	Optimality	Samples
2	FALSE	1178	N/A	N/A	500000
4	FALSE	629	N/A	N/A	200000
5	TRUE	34.84	44	1.375	10000

specifically, even with very long planning horizons (in this case on the order of 500000 iterations), it may still not be possible to find a feasible path.

7) **Heuristic Variation:** Up until this point, for A*-family algorithms, the Euclidean distance was used as a heuristic. To analyze the effects of that choice, this section presents varying heuristic choices for the Weighted A* and ARA* algorithms on the static environments. The parameters used are presented in Table I and the three heuristic choices evaluated are:

- 1) Euclidean Distance (baseline):

$$h_{\text{euc}}(i) = \|\mathbf{x}_i - \mathbf{x}_\tau\|_2 = \sqrt{(x_i - x_\tau)^2 + (y_i - y_\tau)^2 + (z_i - z_\tau)^2} \quad (20)$$

The standard admissible heuristic used throughout this work, with results presents in Table II. Since the straight-line distance is always a lower bound on the true path cost on a 26-connected grid, it never overestimates and guarantees optimality at $\varepsilon = 1$. However, it is not the tightest possible lower bound for this grid structure so it leaves room for unnecessary node expansions.

- 2) 3D Diagonal Distance:

$$h_{\text{oct}}(i) = \delta_3\sqrt{3} + (\delta_2 - \delta_3)\sqrt{2} + (\delta_1 - \delta_2) \quad (21)$$

where $\delta_1 \geq \delta_2 \geq \delta_3$ are the sorted absolute coordinate differences:

$$(\delta_1, \delta_2, \delta_3) = \text{sort}_\downarrow(|x_i - x_\tau|, |y_i - y_\tau|, |z_i - z_\tau|) \cdot \delta \quad (22)$$

This heuristic, which is 3D extension of the octile distance, is a tighter admissible heuristic for a 26-connected uniform grid, as it exactly accounts for the cost structure of diagonal, edge-diagonal, and face-adjacent moves. Since it matches the true grid movement cost more closely than Euclidean distance, it guides the search more precisely toward the goal. Additionally, since it is admissible, it preserves the ε -suboptimality guarantee of Weighted A* and ARA*.

3) Manhattan Distance:

$$h_{\text{man}}(i) = (|x_i - x_\tau| + |y_i - y_\tau| + |z_i - z_\tau|) \cdot \delta \quad (23)$$

The Manhattan distance sums the absolute coordinate differences without accounting for diagonal moves. On a 26-connected grid where diagonal moves cost less than the sum of straight moves that yield the same translation, this heuristic overestimates the true cost-to-go making it inadmissible. Because of this, A*-family algorithms no longer satisfy the guaranteed suboptimality bound. However, this stronger heuristic bias can significantly reduce node expansions and planning time by aggressively directing the search towards the goal. Therefore, the manhattan distance is included here to study the trade-off between computational efficiency and optimality.

Below, Table V describes the results of each A*-family algorithm, for each heuristic, on each environment. The calculated trajectories are omitted because they are largely the same as for the euclidean distance (reference Appendices B-F).

a) Environment 1 (Single Cube): All heuristic choices succeed in this environment and return the optimal path length of 7 with about the same amount of node expansion. It seems that due to this environment's simplicity, heuristic choice has negligible effect. This confirms the theoretical intuition that heuristic quality only matters when the search frontier is large.

b) Environment 2 (Maze): This environment reveals the true difference in performance for each heuristic. For weighted A*, Manhattan expands fewer nodes than euclidean and 3D diagonal distance. This is somewhat counterintuitive because the theory predicts that 3D diagonal distance should be the tightest admissible heuristic leading to the fewest node expansions. This result is likely because in a dense maze, the heuristic guidance is frequently obstructed by obstacle geometry. For ARA*, Manhattan again outperforms 3D diagonal distance, with Manhattan also achieving a slightly worse path length of 82 vs 81 for 3D diagonal distance, reflecting the loss of the optimality guarantee under the inadmissible heuristic.

c) Environment 3 (Flappy Bird): In this environment, the theoretical prediction that 3D diagonal distance should expand the fewest nodes holds for both ARA* and Weighted A*. For Weighted A*, we observe that all three heuristics return the same optimal path, but 3D diagonal distance does so in the

least number of node expansions. For ARA*, 3D diagonal distance achieves a slightly suboptimal path which simply reflects that it terminated at a higher ε under this heuristic compared to the others.

d) Environment 4 (Monza): All heuristics succeed and return the optimal path of 78. Node expansion counts are similar across heuristics, with Manhattan consistently achieving slightly fewer expansions than 3D diagonal distance or Euclidean. Runtimes are also similar (0.83–0.93 s), but are consistently faster for Manhattan distance.

e) Environment 5 (Tower): This environment reveals the most shocking results for differing heuristics. For ARA*, 3D diagonal distance achieves only 364 node expansions in 0.220 s — roughly half that of Manhattan (621 nodes, 0.340 s) and nearly five times fewer than Weighted A* with Euclidean (1626 nodes, 0.754 s). However, ARA* with 3D diagonal distance returns a path of length 39, worse than Manhattan's 33. Because of the tightness of 3D diagonal distance causes ARA* to terminate its first pass very quickly at a high ε , returning a feasible path with fewer expansions, but at lower quality. For weighted A*, both manhattan and 3D diagonal distance return a path of the same length, with similar node counts.

B. Dynamic Environments

For the dynamic environments, the goal moves through 8 positions in a predictable sequence with per-goal time budgets of $t_1 = 3$ s and $t_2 = 1$ s for E6 (Window) and $t_1 = 0.3$ s and $t_2 = 0.1$ s for E7 (Room) in the base case. A goal is successfully reached if the planner returns a collision-free path within the time budget. For a given map, overall success requires that each goal is successfully reached. Due to the static environment results, where sampling-based algorithms failed on several geometrically constrained maps and consistently produced longer paths, only A*-family algorithms are evaluated on the dynamic environments. In this section, each algorithm is pushed to its limit by reducing the time budgets and seeing if success is still feasible.

Below, Table VI describes the parameters used to achieve the results in Table VII. The results should be interpreted a little differently here. The path distance is the sum of the paths from start to each goal. For example, if start to goal 1 had a path length of 25 and start to goal 2 had a path length of 25, the path distance reported would be 50. Similarly, the number of nodes expanded is also the sum. The time budget reflects the amount of time each goal remains in place. This is calculated as $t_1 + 7t_2$ where t_1 is the time the goal remains at position 1 and t_2 is the time the goal remains at each other position.

The paths planned in each environment by each algorithm can be found in Appendices G,H.

1) Environment 6 – Window: Environment 6 has a fairly open environment where the trajectory must traverse through a "window" to a goal that rotates around a circle. Both weighted A* and ARA* succeed.

a) Weighted A:* Weighted A* achieves a minimum total time budget of 0.46 seconds. Across all time budgets, weighted

TABLE V: Performance comparison of heuristic functions for Weighted A* and ARA* across all static environments.

Environment	Algorithm	Heuristic	Success	Runtime (s)	Path Dist.	Nodes Expanded	Optimality
Env. 1	Weighted A*	Manhattan	TRUE	5.62×10^{-3}	7	10	1
	Weighted A*	3D Diagonal Distance	TRUE	6.28×10^{-3}	7	11	1
	ARA*	Manhattan	TRUE	7.34×10^{-3}	7	10	1
	ARA*	3D Diagonal Distance	TRUE	7.51×10^{-3}	7	10	1
Env. 2	Weighted A*	Manhattan	TRUE	3.09	81	6744	1.025
	Weighted A*	3D Diagonal Distance	TRUE	3.411	81	6894	1.025
	ARA*	Manhattan	TRUE	2.895	82	6404	1.04
	ARA*	3D Diagonal Distance	TRUE	3.27	81	6798	1.025
Env. 3	Weighted A*	Manhattan	TRUE	0.379	25	1144	1
	Weighted A*	3D Diagonal Distance	TRUE	0.316	25	825	1
	ARA*	Manhattan	TRUE	0.306	26	888	1.04
	ARA*	3D Diagonal Distance	TRUE	0.222	27	604	1.08
Env. 4	Weighted A*	Manhattan	TRUE	0.839	78	2640	1
	Weighted A*	3D Diagonal Distance	TRUE	0.929	78	2693	1
	ARA*	Manhattan	TRUE	0.831	78	2624	1
	ARA*	3D Diagonal Distance	TRUE	0.892	78	2685	1
Env. 5	Weighted A*	Manhattan	TRUE	0.708	33	1518	1.03
	Weighted A*	3D Diagonal Distance	TRUE	0.723	33	1459	1.03
	ARA*	Manhattan	TRUE	0.340	33	621	1.03
	ARA*	3D Diagonal Distance	TRUE	0.220	39	364	1.22

TABLE VI: Parameters used for all dynamic Environments.

Algorithm	Parameter	Value	Description
Weighted A*	ε	1.5	Heuristic inflation weight
	δ	0.5	Grid resolution (m)
ARA*	$\varepsilon_{\text{init}}$	5.0	Initial heuristic weight
	$\varepsilon_{\text{final}}$	1.0	Final heuristic weight
	$\Delta\varepsilon$	0.5	Weight reduction step
	T_{limit}	5.0s	Anytime time budget
	δ	0.5	Grid resolution (m)

A* returns the same path length and node expansion count because it does not have anytime behavior, so it does not end its search early ever. As long as the time budget exceeds the fixed per goal planning time, the algorithm succeeds.

b) *ARA**: ARA* achieves a minimum total time budget of 0.39 seconds, which displays the advantage of the anytime behavior. ARA* consistently returns a path length of 178.120 with 630 nodes expanded, a suboptimality ratio of 1.044 relative to weighted A*. The reduced node count for ARA* reflects ARA*'s termination as soon as a feasible solution is found under $\varepsilon_{\text{init}} = 5.0$.

Crucially, ARA* reuses g -values across ε -reduction iterations within each planning call. When ε is reduced from ε_k to ε_{k+1} , the `g_score` and `parent` dictionaries are not reinitialised — only the `closed` set and `OPEN` list are reset. Nodes whose cost estimates are already consistent under the tighter bound are settled immediately upon re-expansion without re-traversing the path from the start. In Environment 6, this means that each successive ε -reduction pass re-expands only the inconsistent frontier rather than the full 630 nodes, making path quality improvement within the time budget computationally cheap.

2) *Environment 7 – Room*: In comparison to Environment 6, Environment 7 has a much tighter time budget. The room

environment also contains many walls and is much more geometrically constrained. Additionally, the goals are scattered throughout the room.

a) *Weighted A**: Weighted A* succeeds at the base case, returning a path length of 69.753 with 604 nodes. However, it could not succeed with a severely lower time budget and its minimum time budget was 0.352 seconds. This reflects the fact that the full search must be completed in order to guarantee a solution, enforcing the intuition that Weighted A* is an all-or-nothing algorithm. This is a fundamental limitation in dynamic settings – if the goal is not reached within the time budget, nothing is returned.

b) *ARA**: ARA* also fails at the severely reduced budget of 0.2 seconds, but its minimum budget of 0.306 seconds is lower than Weighted A*, again demonstrating the anytime behavior advantage. ARA* returns a path length of 70.629 (ratio 1.013 relative to Weighted A*) with only 494 nodes expanded. The reduction in node expansions relative to Weighted A* is again explained by early termination under high ε .

c) *Overall Observations*: ARA* consistently achieves lower time budgets across both environments while still returning paths of similar quality. The path quality and node expansion counts are consistent for both algorithms when they succeed, which confirms that both complete their search within the available time in all non-failure cases. When the time budget is severely reduced, ARA*'s anytime property delays the failure threshold, making it the better choice for dynamic motion planning settings.

TABLE VII: Performance comparison of A*-family planning algorithms in dynamic environments.

Environment	Algorithm	Success	Time Budget = $t_1 + 7t_2$ (s)	Path Dist.	Nodes
Env. 6 - BASE CASE	Weighted A*	TRUE	10.0	170.635	858
	ARA*	TRUE	10.0	178.120	630
Env. 6 - Slightly Lower Time Budget	Weighted A*	TRUE	8.0	170.63	858
	ARA*	TRUE	8.0	178.120	630
Env. 6 - Moderately Lower Time Budget	Weighted A*	TRUE	4.0	170.635	858
	ARA*	TRUE	4.0	178.120	630
Env. 6 - Severely Lower Time Budget	Weighted A*	TRUE	2.0	170.635	858
	ARA*	TRUE	2.0	178.120	630
Env. 6 - Minimum Time Budget Achievable	Weighted A*	TRUE	0.46	170.635	858
	ARA*	TRUE	0.39	178.120	630
Env. 7 - BASE CASE	Weighted A*	TRUE	1.0	69.753	604
	ARA*	TRUE	1.0	70.629	494
Env. 7 - Slightly Lower Time Budget	Weighted A*	TRUE	0.8	69.753	604
	ARA*	TRUE	0.8	70.629	494
Env. 7 - Moderately Lower Time Budget	Weighted A*	TRUE	0.4	69.753	604
	ARA*	TRUE	0.4	70.629	494
Env. 7 - Severely Lower Time Budget	Weighted A*	FALSE	0.2	N/A	N/A
	ARA*	FALSE	0.2	N/A	N/A
Env. 7 - Minimum Time Budget Achievable	Weighted A*	TRUE	0.352	69.753	604
	ARA*	TRUE	0.306	70.629	494

V. CONCLUSION

In this work, we compared ARA*, Weighted A*, RRT*, and RRT-connect across a range of 3D static and dynamic environments. All algorithms used the same collision checking framework based on the slab method for axis-aligned bounding boxes.

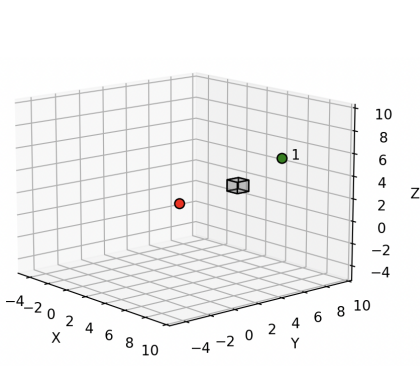
The results reflect the tradeoffs in motion planning, particularly when choosing between sampling- and search-based algorithms. Search-based methods consistently performed better in environments with tightly constrained geometry, while sampling-based methods, namely RRT-connect, performed extremely fast in more open environments.

Overall, the experiments demonstrate that no single planning algorithm is always optimal. Among all evaluated methods, ARA* balanced speed, robustness, and path quality the best.

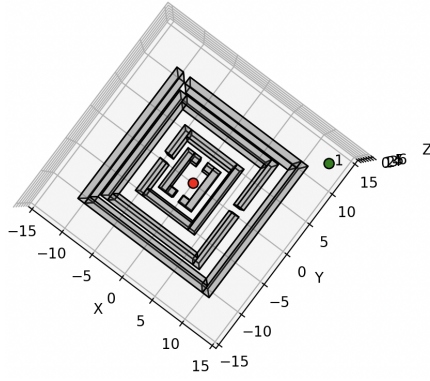
REFERENCES

- [1] N. Atanasov, “Ece276b: Planning & learning in robotics — lecture 6: Configuration space,” 2025. Course lecture notes.
- [2] N. Atanasov, “Ece276b: Planning & learning in robotics — lecture 5: Deterministic shortest path,” 2025. Course lecture notes.
- [3] T. L. Kay and J. T. Kajiya, “Ray tracing complex scenes,” in *Computer Graphics (SIGGRAPH '86 Proceedings)*, vol. 20, pp. 269–278, ACM, 1986.
- [4] N. Atanasov, “Ece276b: Planning & learning in robotics — lecture 7: Search-based motion planning,” 2025. Course lecture notes.
- [5] M. Likhachev, G. Gordon, and S. Thrun, “Ara*: Anytime a* with provable bounds on sub-optimality,” in *Advances in Neural Information Processing Systems*, 2004.
- [6] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.
- [7] J. J. Kuffner and S. M. LaValle, “Rrt-connect: An efficient approach to single-query path planning,” in *IEEE International Conference on Robotics and Automation*, 2000.

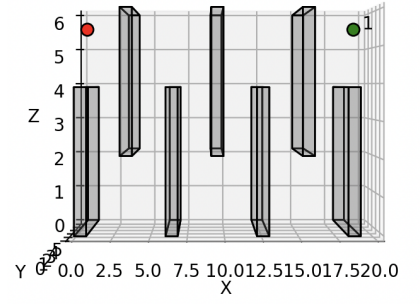
APPENDIX A VISUALIZATION OF ENVIRONMENTS



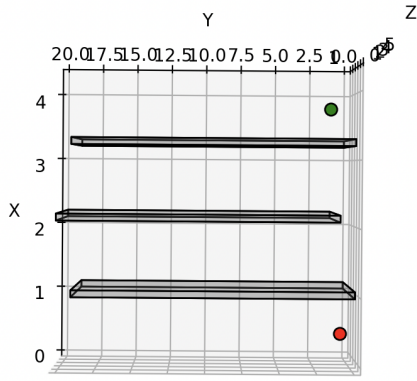
(a) Env. 1 – “Single Cube”



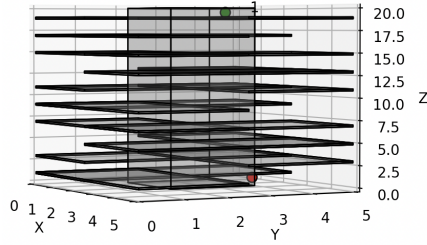
(b) Env. 2 – “Maze”



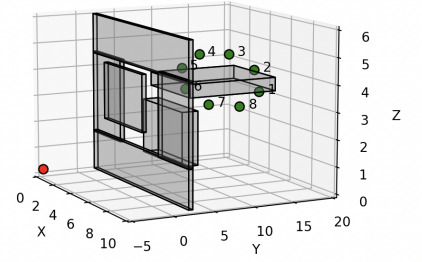
(c) Env. 3 – “Flappy Bird”



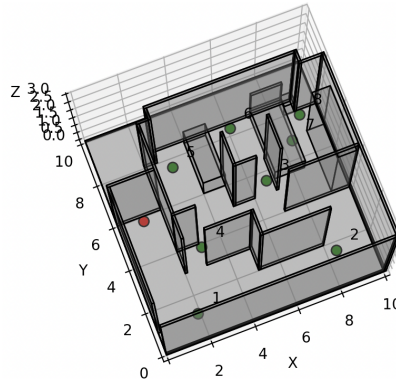
(d) Env. 4 – “Monza”



(e) Env. 5 – “Tower”



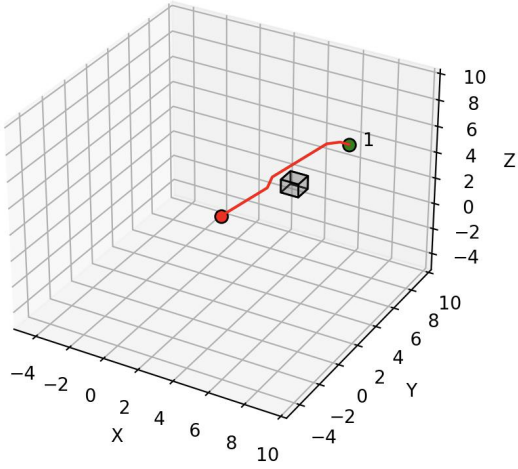
(f) Env. 6 – “Window”



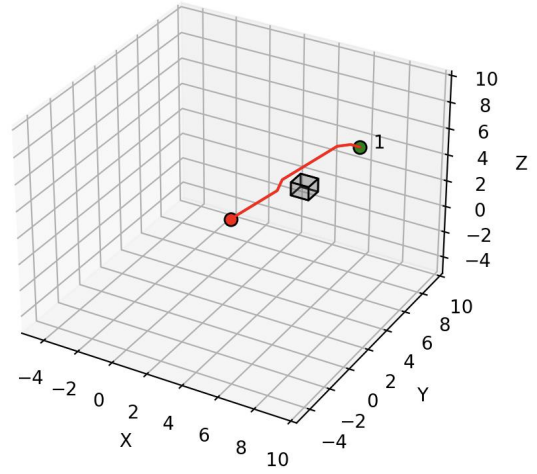
(g) Env. 7 – “Room”

Fig. 1: Environments E1–E7. The start is the red dot and the goal is the green dot. E1–E5 are static. In E6–E7, the goal rotates between the green dots.

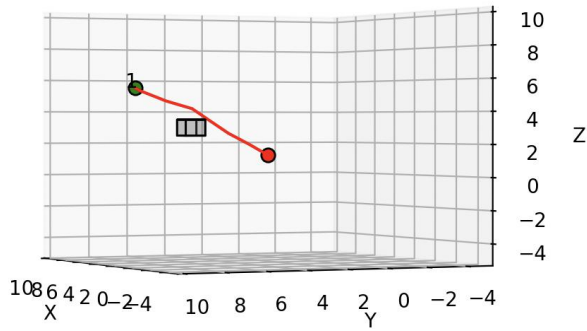
APPENDIX B
ENVIRONMENT 1 – “SINGLE CUBE”



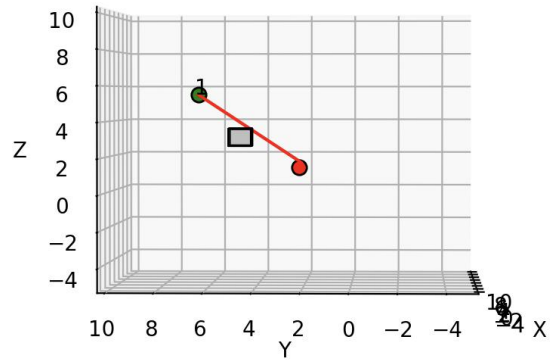
(a) Weighted A* on env 1



(b) ARA* on env 1



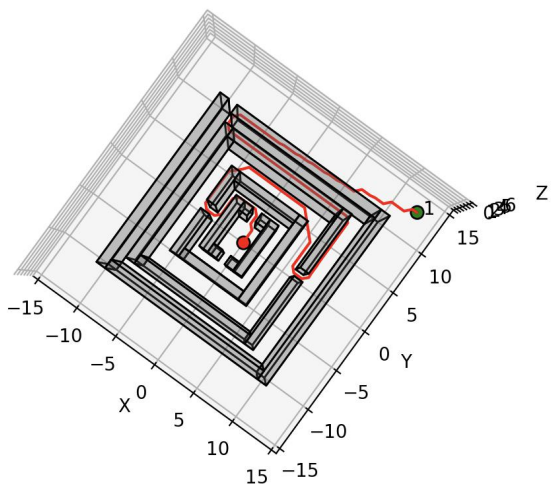
(c) RRT* on env 1



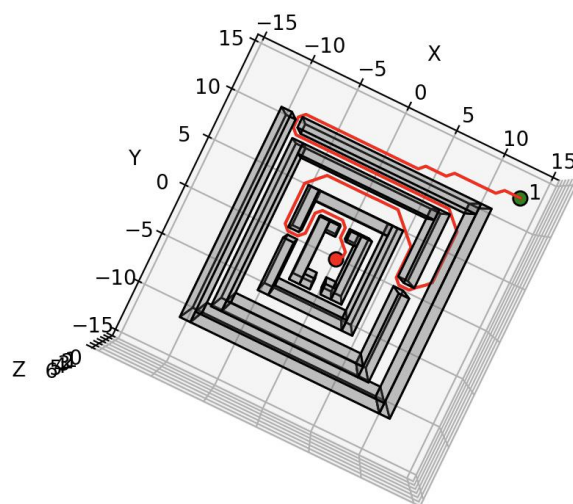
(d) RRT Connect on env 1

Fig. 2: All four algorithm example trajectories on Environment 1. All four algorithms succeed with ease.

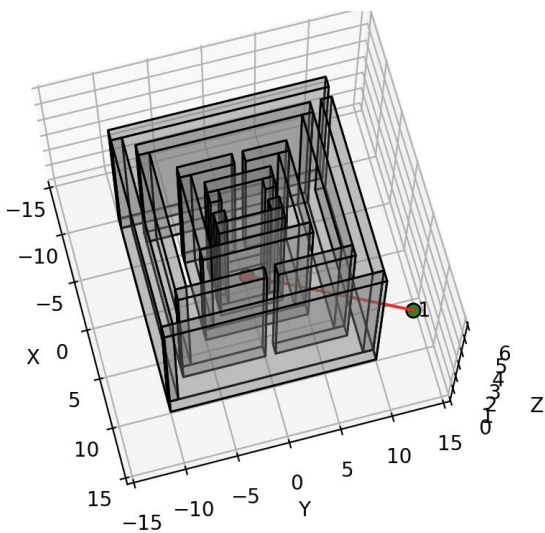
APPENDIX C
ENVIRONMENT 2 – “MAZE”



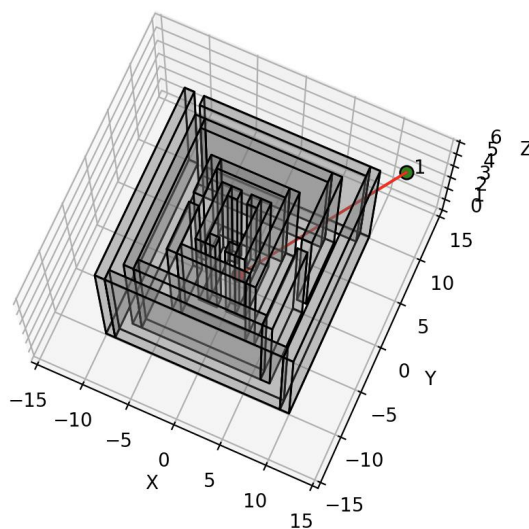
(a) Weighted A* on env 2



(b) ARA* on env 2



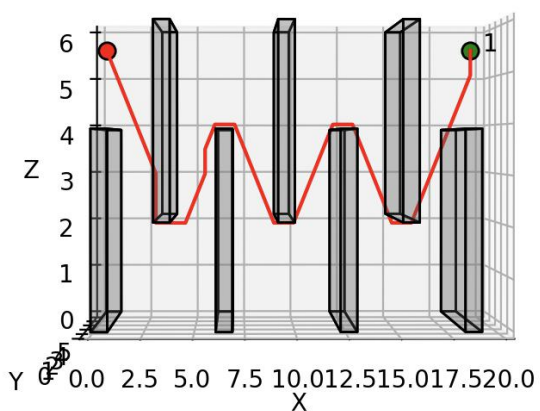
(c) RRT* on env 2



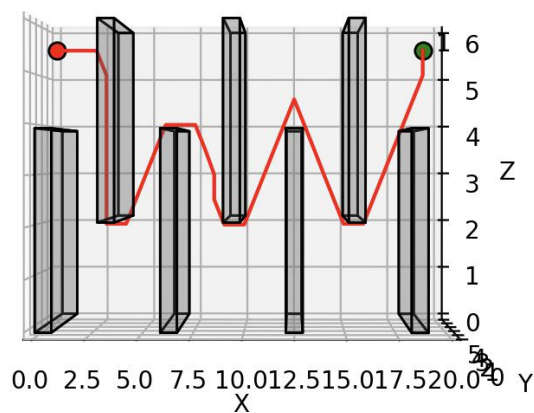
(d) RRT Connect on env 2

Fig. 3: All four algorithm example trajectories on Environment 2. Sampling-based algorithms default to straight line due to failure, reflecting the narrow passage problem.

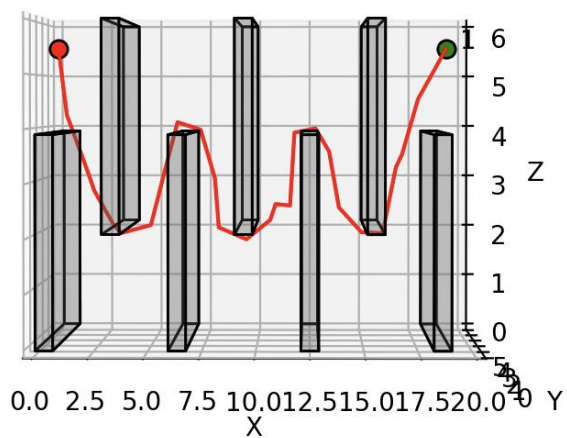
APPENDIX D
ENVIRONMENT 3 – “FLAPPY BIRD”



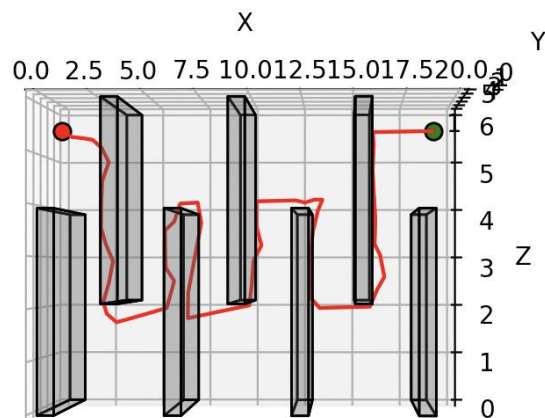
(a) Weighted A* on env 3



(b) ARA* on env 3



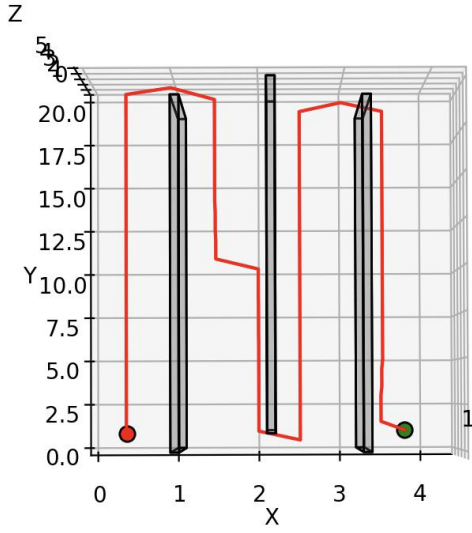
(c) RRT* on env 3



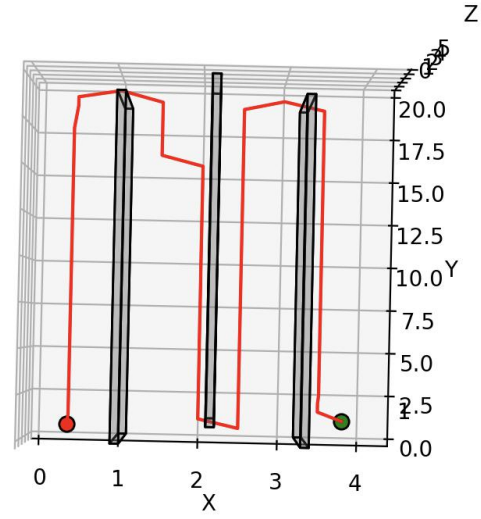
(d) RRT Connect on env 3

Fig. 4: All four algorithm example trajectories on Environment 3. All four algos succeed.

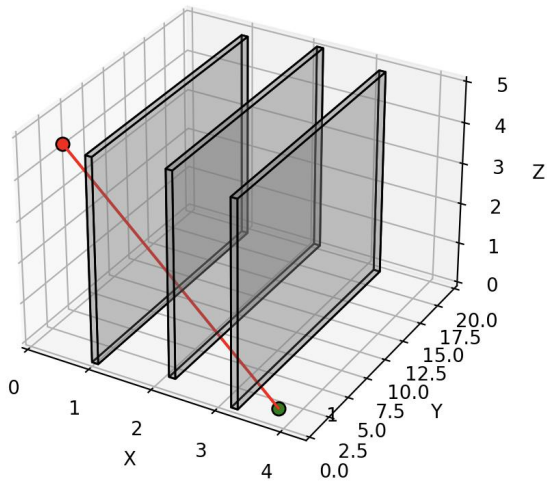
APPENDIX E
ENVIRONMENT 4 – “MONZA”



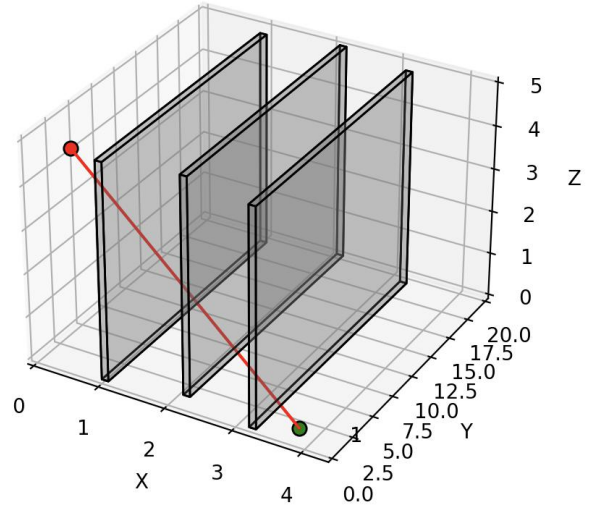
(a) Weighted A* on env 4



(b) ARA* on env 4



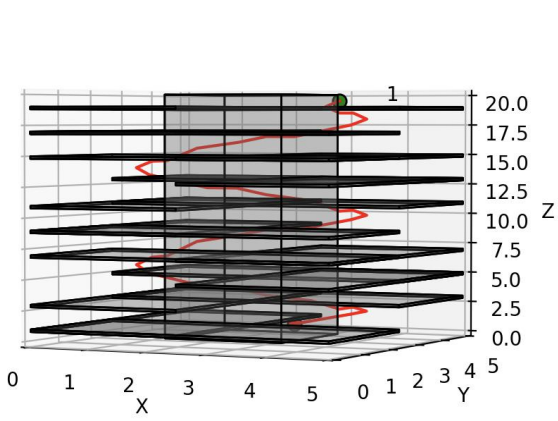
(c) RRT* on env 4



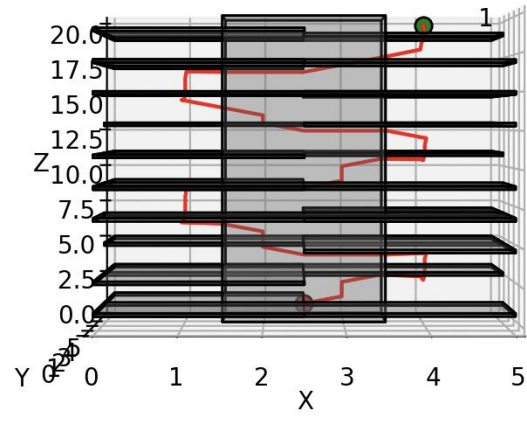
(d) RRT Connect on env 4

Fig. 5: All four algorithm example trajectories on Environment 4. Sampling-based algos fail due to the narrow passages.

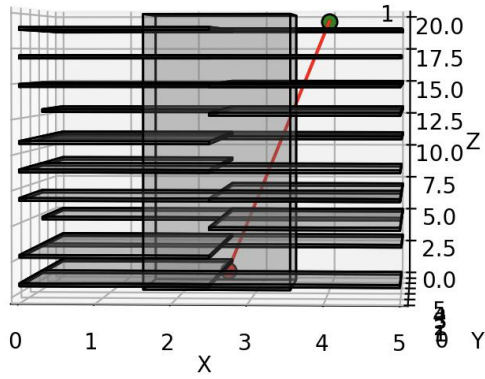
APPENDIX F
ENVIRONMENT 5 – “TOWER”



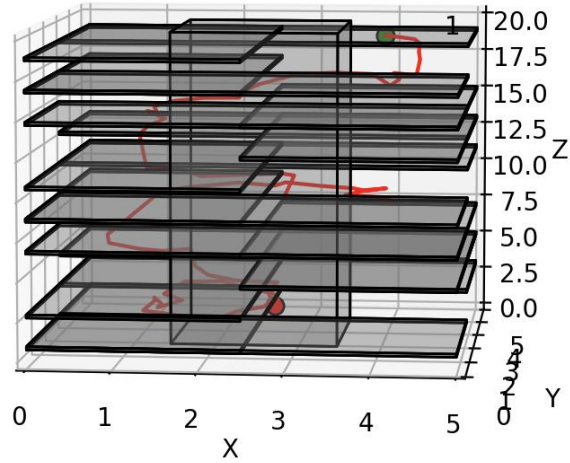
(a) Weighted A* on env 5



(b) ARA* on env 5



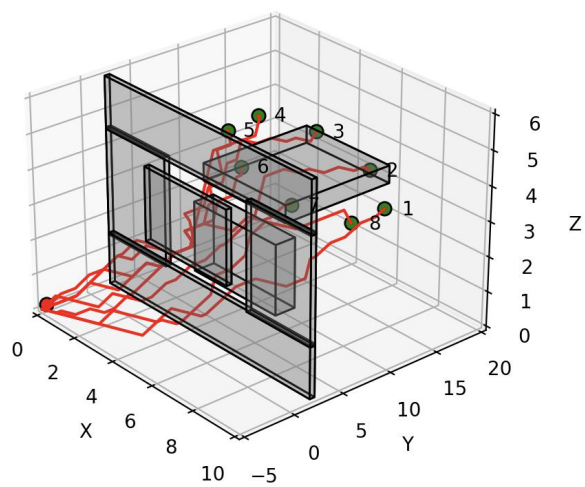
(c) RRT* on env 5



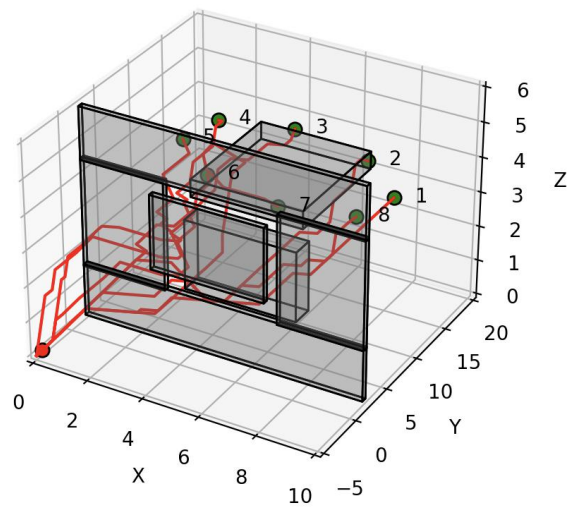
(d) RRT Connect on env 5

Fig. 6: All four algorithm example trajectories on Environment 5. Only RRT* fails, showing the advantage of bidirectional tree growth.

APPENDIX G
ENVIRONMENT 6 – “WINDOW”



(a) Weighted A* on env 6



(b) ARA* on env 6

Fig. 7: A*-family algorithm example trajectories on Environment 6. Both algorithms succeed, but ARA* is faster.

APPENDIX H
ENVIRONMENT 7 – “ROOM”

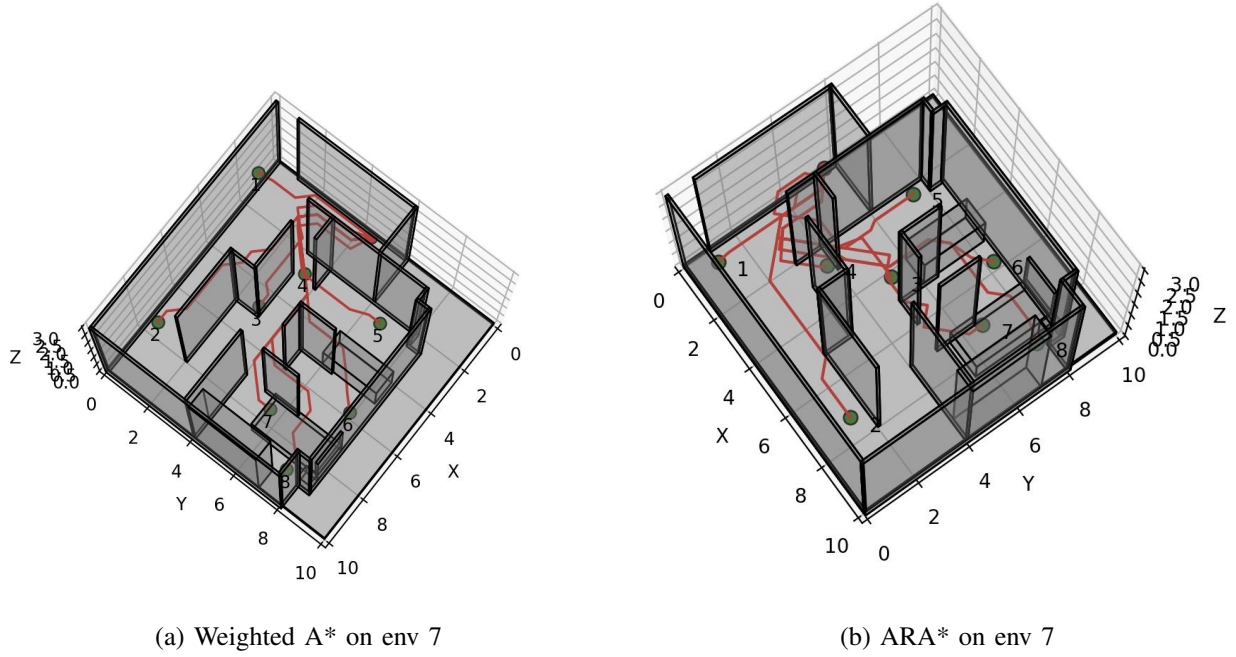
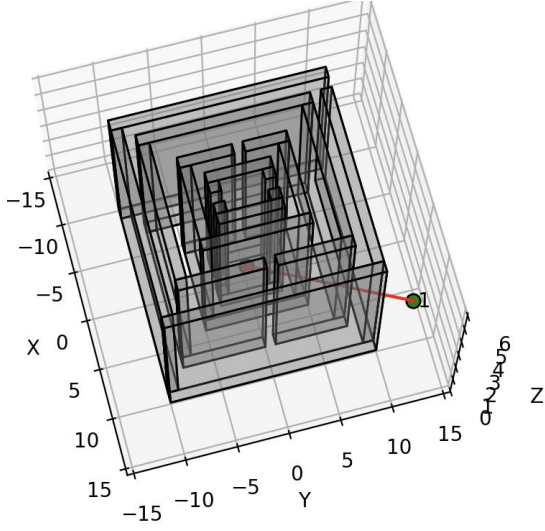
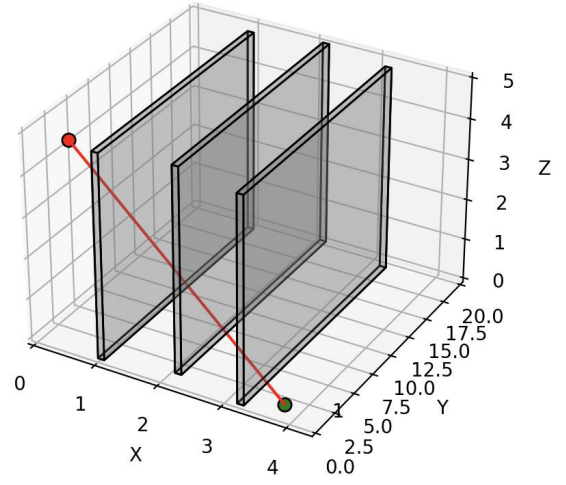


Fig. 8: A*-family algorithm example trajectories on Environment 7. Both algorithms succeed, but ARA* is faster.

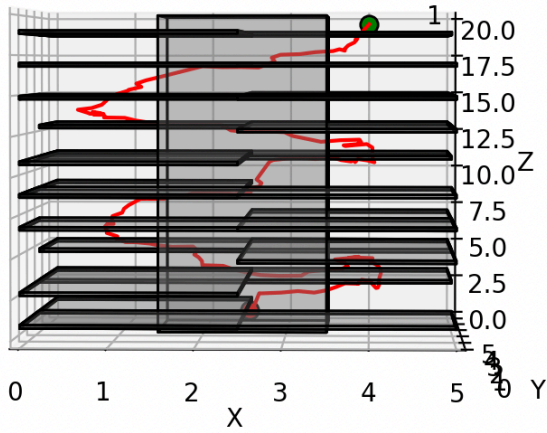
APPENDIX I
RRT* WITH VARIED PARAMETERS



(a) RRT* with 500000 iterations on env 2



(b) RRT* with 200000 iterations on env 4



(c) RRT* with 10000 iterations on env 5

Fig. 9: Trajectories of RRT* in narrow passage problems with varied parameters.